Union Find

What is Union Find?

Union Find (also called Disjoint Set Union, DSU) maintains a dynamic family of disjoint sets over elements $\{0, 1, \dots, n-1\}$.

It supports three core operations from the lecture:

- `INIT(n)`: create singleton sets $\{0\}, \{1\}, \dots, \{n-1\}$
- `UNION(i,j)`: merge the sets containing `i` and `j`
- `FIND(i)`: return a representative of the set containing `i`

If `i` and `j` are already in the same set, `UNION(i,j)` does nothing.

Why It Matters

Typical applications include:

- Dynamic connectivity in graphs
- [[Minimum Spanning Tree (MST)]] (especially Kruskal)
- Other equivalence/merging problems where components evolve over time

Quick Find

Idea

Store an array `id[0..n-1]` where `id[i]` is the representative label for element `i`.

- `FIND(i)` is immediate: return `id[i]`
- `UNION(i,j)` scans the full array and relabels one component to the other

Pseudocode

```
INIT(n)
    for k = 0 to n-1:
        id[k] = k

FIND(i)
    return id[i]

UNION(i, j)
    iID = FIND(i)
    jID = FIND(j)
    if iID != jID:
        for k = 0 to n-1:
            if id[k] == iID:
                id[k] = jID
```

Running Time

From the slides:

- `INIT`: $O(n)$
- `UNION`: $O(n)$
- `FIND`: $O(1)$

Quick Union

![[quick-union-example.png]] ### Idea Represent each set as a rooted tree using parent array `p[0..n-1]`.

- `p[i]` is parent of `i`
- root nodes satisfy `p[root] = root`
- representative is the root

`FIND(i)` follows parent pointers to the root. `UNION(i,j)` links one root under the other root.

Pseudocode

```
INIT(n)
    for k = 0 to n-1:
        p[k] = k

FIND(i)
    while i != p[i]:
        i = p[i]
    return i

UNION(i, j)
    ri = FIND(i)
    rj = FIND(j)
    if ri != rj:
        p[ri] = rj
```

Running Time

Let d be tree depth.

- `INIT`: $O(n)$
- `FIND`: $O(d)$
- `UNION`: $O(d)$

Worst case: depth can become $n-1$, so operations can degrade to linear time.

![[quick-union-example-2.png]]

Weighted Quick Union

Idea

Improve Quick Union by always attaching the smaller tree under the larger tree.

Maintain extra array `sz[0..n-1]` where `sz[r]` is subtree size at root `r`.

Pseudocode

```
INIT(n)
    for k = 0 to n-1:
        p[k] = k
        sz[k] = 1

FIND(i)
    while i != p[i]:
        i = p[i]
    return i

UNION(i, j)
    ri = FIND(i)
    rj = FIND(j)
    if ri != rj:
        if sz[ri] < sz[rj]:
            p[ri] = rj
            sz[rj] = sz[rj] + sz[ri]
        else:
            p[rj] = ri
            sz[ri] = sz[ri] + sz[rj]
```

![[weighted-quick-union-example.png]]

Key Lemma (from lecture)

With weighted quick union, node depth is at most $\log_2 n$.

Reason: depth of a node increases only when its tree is attached under a larger tree, so the size of its containing tree at least doubles each time. A set can double at most $\log_2 n$ times.

Running Time

- `INIT`: $O(n)$
- `FIND`: $O(\log n)$
- `UNION`: $O(\log n)$

Path Compression

Idea

During `FIND(i)`, after locating the root, make all visited nodes point directly to the root.

This does not significantly speed up a single operation, but it dramatically improves future operations.

Works with both Quick Union and Weighted Quick Union.

Pseudocode (two-pass style)

```
FIND(i)
    r = i
    while r != p[r]:
        r = p[r]

    while i != r:
        next = p[i]
        p[i] = r
        i = next

    return r
```

Theoretical Guarantee

Tarjan (1975): any sequence of m `FIND/UNION` operations on n elements takes

$$O(n + m\,\alpha(m, n)),$$

where α is inverse Ackermann, which is at most 5 for practical inputs.

So amortized cost per operation is effectively near-constant.

Dynamic Connectivity

Problem

Maintain a dynamic undirected graph with:

- `INIT(n)`: graph with `n` vertices and no edges
- `INSERT(u,v)`: add edge (u, v)
- `CONNECTED(u,v)`: test whether `u` and `v` are in the same connected component

Reduction to Union Find

Use one Union Find structure over vertices:

- `INIT(n)` -> initialize Union Find with `n` singleton sets
- `INSERT(u,v)` -> `UNION(u,v)`
- `CONNECTED(u,v)` -> `FIND(u) == FIND(v)`

With weighted quick union, this gives:

- `INIT`: $O(n)$
- `INSERT`: $O(\log n)$
- `CONNECTED`: $O(\log n)$

With path compression, operation sequences are even faster in amortized terms.

Summary Table

Structure	`UNION`	`FIND`
Quick Find	$O(n)$	$O(1)$
Quick Union	$O(n)$ worst case	$O(n)$ worst case
Weighted Quick Union	$O(\log n)$	$O(\log n)$
Weighted Quick Union + Path Compression	near-constant amortized	near-constant amortized

Related

- [[Minimum Spanning Tree (MST)]]
- [[Graphs]]
- [[Amortized Analysis]]